Technical report · hands-on-policy-distillation

On-Policy Distillation from First Principles

Two small Qwen models learn to use tools

October 2026 · Qwen3-8B → Qwen3-1.7B and Qwen3.5-9B → Qwen3.5-2B on one A100-80GB

Abstract

We distil Qwen3-8B into Qwen3-1.7B and Qwen3.5-9B into Qwen3.5-2B with the same code and hyperparameters on one A100-80GB GPU, in four stages: grade-school math, a calculator tool, three tools over multi-turn episodes, and distillation plus a task reward. Starting from what the student should imitate, and where, we derive on-policy distillation as a policy gradient whose per-token reward is the teacher's log-probability of the student's token minus the student's own. Teacher KL falls in every distillation stage, and task success rises although no reward is used: from 0.47 to 1.00 and from 0.65 to 0.90 on calculator tasks, from 0.66 to 0.75 and from 0.75 to 0.96 on three-tool tasks. Adding the task reward through GRPO changes nothing measurable, because 98.5% and 89% of sample groups receive identical rewards. The Qwen3.5 run takes 2.5 times the GPU time, because its episodes are 2–5 times longer.

0.47 → 1.00
calculator tasks solved by the Qwen3 student, with no reward

0.75 → 0.96
three-tool tasks solved by the Qwen3.5 student, with no reward

98.5%
of Qwen3's GRPO groups had identical rewards: no signal

2.5×
the GPU time for Qwen3.5, whose episodes are 2–5 times longer

1 Introduction

A small model that calls tools is cheap to serve, but it has to learn when to call which tool, how to write the call, and what to do with the result. There are three ways to teach it: supervised fine-tuning on episodes that a larger teacher wrote (off-policy distillation; Kim and Rush, 2016); reinforcement learning from a task reward, for example with GRPO (Shao et al., 2024); and on-policy distillation, in which the student writes the episodes and the teacher grades every token (Agarwal et al., 2024; Gu et al., 2024; Lu and Thinking Machines Lab, 2025). The small Qwen3 models were themselves post-trained with on-policy distillation from larger ones (Yang et al., 2025a).

This report explains from first principles why the third works (Section 2), how we trained (Section 3), and what two runs showed (Section 4). Both runs use one pipeline (TRL's GRPO trainer with a teacher term added to the advantage) and the same configs; only the models differ.

2 The core idea from first principles

Setting. A language model is a policy over tokens. The state s_t is everything in the context before the student's t-th token: the prompt (system prompt, tool descriptions, task), the student's earlier tokens, and the results of the tools it has called. The action is the next token y_t. The student π_θ is trained; the teacher π_T is frozen. Tool results are written by the environment, not chosen by the student, so they are part of the state and never an action.

We want the student to choose tokens, in every state, the way the teacher would. The natural discrepancy between two next-token distributions is a KL divergence, which leaves two choices: in which states to measure it, and in which direction.

Which states: the student's own. Fine-tuning on teacher-written episodes trains the student on the states the teacher visits. At test time the student generates, and visits its own. After one token the teacher would not have written, the student is in a state that no training example covered, where it is more likely to err again: errors compound. In imitation learning with per-step error ϵ on the training distribution, the cost over T steps can grow as $O(\epsilon T^2)$ (Ross and Bagnell, 2010); training on states drawn from the learner's own policy, labelled by an expert, brings it down to $O(\epsilon T)$ (DAgger; Ross et al., 2011). On-policy distillation is this recipe with the teacher as the expert: the student writes the episodes, and the teacher says what it would have done in every state the student reached.

Which direction: reverse KL. The objective is the reverse KL, summed over the states of the student's own episodes τ:

$$\mathscr{L}(\theta) = \mathbb{E}_x \, \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t \mathrm{KL}\big(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_T(\cdot \mid s_t)\big)\Big]. \tag{1}$$

By the chain rule of KL, this expected sum is the KL between the two models' distributions over whole episodes; tool results are the same function of the history under both models and cancel. Reverse KL suits distillation into a small model for three reasons. (i) It is an expectation under the student, so it can be estimated from the student's samples and the teacher's log-probabilities of those tokens: one teacher forward pass, no teacher generation. (ii) It is mode seeking: it punishes the student hard for putting probability where the teacher puts almost none, and mildly for leaving out some of the teacher's options. A student that cannot represent everything a larger teacher does commits to one good behaviour instead of averaging several (Gu et al., 2024; Lu and Thinking Machines Lab, 2025). (iii) It reaches its minimum, zero, only when the student behaves like the teacher in the states it visits, so nothing but imitation lowers it. Unlike a learned reward model, it cannot be gamed; Lu and Thinking Machines Lab (2025) call it unhackable.

The gradient is a policy gradient. Fix a state s and write $p = \pi_\theta(\cdot \mid s)$ and $q = \pi_T(\cdot \mid s)$, with a ranging over the vocabulary. Since $p(a)\nabla_\theta \log p(a) = \nabla_\theta p(a)$ and $\sum_a \nabla_\theta p(a) = \nabla_\theta 1 = 0$,

$$\begin{aligned}\nabla_\theta \mathrm{KL}(p \,\|\, q) &= \sum_a \nabla_\theta p(a)\big[\log p(a) - \log q(a)\big] + \sum_a \nabla_\theta p(a) \\ &= \mathbb{E}_{a\sim p}\Big[\big(\log p(a) - \log q(a)\big)\,\nabla_\theta \log p(a)\Big].\end{aligned} \tag{2}$$

Define, for the token y_t the student actually sampled,

$$k_t = \log \pi_\theta(y_t \mid s_t) - \log \pi_T(y_t \mid s_t). \tag{3}$$

By (2), $k_t \, \nabla_\theta \log \pi_\theta(y_t \mid s_t)$ is an unbiased one-sample estimate of the gradient of the KL at s_t. That is the whole method:

Core identity

Minimizing the reverse KL (1) on the student's own samples is REINFORCE (Williams, 1992): minimize the surrogate loss

$$\ell(\theta) = -\frac{1}{N} \sum_t A_t \, \log \pi_\theta(y_t \mid s_t), \qquad A_t = -k_t \;\text{(held constant)}, \tag{4}$$

over the N student tokens of a batch. The reward of token t is

$$r_t = \log \pi_T(y_t \mid s_t) - \log \pi_\theta(y_t \mid s_t). \tag{5}$$

Algorithm 1 One training step

Require: student π_θ (LoRA), frozen teacher π_T, environment, prompts $x_1, \ldots, x_P$

for each prompt x_p **do**
  sample G episodes with π_θ (vLLM), running every tool call and appending its result
  $R_{p,i} \leftarrow$ task reward of episode i; $\hat{A}_{p,i} \leftarrow (R_{p,i} - \bar{R}_p)/\sigma_p$ ▷ 0 when $G = 1$
end for
teacher forward pass over every episode: $k_t \leftarrow \log \pi_\theta(y_t \mid s_t) - \log \pi_T(y_t \mid s_t)$
$A_t \leftarrow c_R \hat{A}_{p,i} - c_T k_t$ for every student token t of episode (p, i)
one AdamW step on $\ell(\theta) = -\frac{1}{N} \sum_t A_t \log \pi_\theta(y_t \mid s_t)$ ▷ N student tokens

Stages differ only in the environment, the group size G, and (c_T, c_R).

The reward is the teacher's log-probability of what the student wrote plus the student's own surprisal (an entropy bonus). Tokens the teacher likes more than the student did are pushed up, tokens it likes less are pushed down. Averaged over tokens, k_t is also an unbiased estimate of the per-token reverse KL; it is the teacher-KL curve in Figure 1.

Two approximations. First, $\mathscr{L}$ depends on θ also through the states, because every token shapes the states that follow; the exact gradient of (1) adds the future terms $\sum_{t'>t} k_{t'}$ to A_t. Like Lu and Thinking Machines Lab (2025), we drop them (a discount factor of zero): each token answers only for its own disagreement, which lowers variance, and the next batch trains on whatever states the updated student visits. Second, one sampled token stands in for the sum over the vocabulary in (2). GKD (Agarwal et al., 2024) computes that sum exactly, which needs both models' full next-token distributions at every position (248,320 numbers per position for Qwen3.5); the sampled estimate needs one number from the teacher.

Why a dense signal beats a sparse one. Compare RL from a binary task reward $R \in \{0, 1\}$. GRPO samples a group of G episodes per prompt and gives every token of episode i the same advantage $\hat{A}_i = (R_i - \bar{R})/\sigma_R$, normalized within the group. Two things follow. An episode's reward carries at most one bit, spread over hundreds of tokens: it says that the episode failed, not where. And if all G episodes of a prompt get the same reward, every advantage in the group is zero. If the student solves a prompt with probability p, independently across samples, a group carries a signal with probability

$$P_{\text{signal}}(p) = 1 - p^G - (1 - p)^G, \tag{6}$$

which for $G = 4$ is 0.875 at $p = \frac{1}{2}$, 0.17 at $p = 0.955$, and vanishes as p approaches 0 or 1. The teacher term instead gives one number per token, $O(T)$ bits per episode rather than $O(1)$ (Lu and Thinking Machines Lab, 2025), and it is nonzero wherever student and teacher disagree, whether or not the episode succeeded. It is also cheap: the teacher scores an episode in one forward pass, parallel over positions, and never decodes.

Combining both signals. In general, the advantage of token t of episode i is

$$A_t = c_R \hat{A}_i - c_T k_t, \tag{7}$$

with $c_R = 0$ for pure distillation. The teacher term can at best make the student as good as the teacher; only the task reward can push past the teacher or fix its mistakes, and only through the groups that (6) leaves informative. Algorithm 1 is one training step of every stage.

Table 1 The two teacher–student pairs. Everything else is identical.

	Qwen3 run	Qwen3.5 run
Teacher (frozen, bf16)	Qwen3-8B	Qwen3.5-9B (language model only)
Student	Qwen3-1.7B	Qwen3.5-2B
Student layers	28, all attention	18 Gated DeltaNet + 6 gated attention
Vocabulary	151,936	248,320
Tool-call syntax	JSON	XML
Trainable LoRA parameters	34.9 M	33.6 M

Table 2 The curriculum, identical in both runs. Tokens: limit on the whole episode after the prompt, tool results included. Batch: episodes per step, prompts × group size G.

Stage	Tasks	Tools	Turns	Tokens	Steps	Batch	LR	(c_T, c_R)
1	GSM8K	none	1	1,024	200	32×1	10^{-4}	(1, 0)
2	big-number arithmetic	calculator	4	2,048	150	32×1	10^{-4}	(1, 0)
3	one, two or no tools	all three	6	3,072	200	32×1	10^{-4}	(1, 0)
4	as stage 3	all three	6	3,072	150	16×4	$5 \cdot 10^{-5}$	(1, 1)

Preconditions. k_t compares two models' probabilities of the same token id, so student and teacher need the same tokenizer: Qwen3 (151,936 tokens) and Qwen3.5 (248,320) cannot be mixed, which is why each family is distilled within itself. Prompts and tool results enter s_t but carry no loss. And the teacher has to judge states it would never reach on its own, such as those after a student's mistake.

3 Training setup

Models and hardware. Table 1 lists the two teacher–student pairs. Qwen3.5 (Qwen Team, 2026) interleaves Gated DeltaNet layers (Yang et al., 2025b), a form of linear attention, with gated attention layers, three to one. Both runs use non-thinking mode, LoRA (Hu et al., 2022) of rank 32 and $\alpha = 32$ on every linear layer of the student's language model (the Qwen3.5 checkpoints also carry a vision tower, which these text-only tasks leave unused), and one Colab A100-SXM4-80GB. The trainer is TRL's `GRPOTrainer` (von Werra et al., 2020) with vLLM generation (Kwon et al., 2023). Our subclass computes k_t after each rollout batch and replaces GRPO's advantage with (7); sampling, tool execution, masking and the loss are TRL's.

Curriculum. Table 2 lists the four stages. Stage 1 samples GSM8K questions (Cobbe et al., 2021); their reference answers never enter the loss. Stage 2's word problems have numbers large enough (a five-digit times a four-digit number, say) that arithmetic in prose often goes wrong. Stages 3 and 4 offer three tools on every task: a calculator, read-only SQL over a randomly generated company database, and JMESPath queries over a randomly generated store document. Their tasks need one tool (45%), two tools combined (40%), or none (15%, to be answered directly). The tasks of stages 2–4 are generated, with different random seeds for training and evaluation. Each stage keeps training the previous stage's LoRA adapter.

Optimization and scoring.

- Every optimizer step uses a fresh batch sampled by the current weights, so the PPO ratio is one, clipping ($\epsilon = 0.2$ below, 0.28 above) never binds, and the update is the gradient of (4), averaged over all student tokens of the batch (TRL's DAPO loss; Yu et al., 2025).
- AdamW with a constant learning rate after 5 warm-up steps, gradient norm clipped at 1, no weight decay; micro-batches of 8 episodes with gradient checkpointing. A rate of 10^{-4} is about ten times a full fine-tuning rate, as recommended for LoRA (Schulman and Thinking Machines Lab, 2025).

Table 3 Evaluation at the start and end of each stage (256 held-out tasks, temperature 1), and wall-clock time including evaluation.

	Qwen3-8B → Qwen3-1.7B			Qwen3.5-9B → Qwen3.5-2B		
Stage	Teacher KL	Reward	Time	Teacher KL	Reward	Time
1. GSM8K	0.229 → 0.136	0.809 → 0.836	45 min	0.218 → 0.168	0.656 → 0.781	85 min
2. Calculator	0.328 → 0.057	0.473 → 1.000	18 min	0.108 → 0.047	0.645 → 0.902	103 min
3. Three tools	0.378 → 0.068	0.660 → 0.746	52 min	0.163 → 0.034	0.754 → 0.957	109 min
4. + Task reward	0.065 → 0.067	0.754 → 0.758	55 min	0.034 → 0.025	0.945 → 0.953	130 min
Total			2.8 h			7.1 h

- Sampling: vLLM on the training GPU (30% of its memory), temperature 1, top-p 1.
- Scoring: one bf16 teacher forward pass per episode, through a fused output layer that returns the log-probabilities of the sampled tokens without materializing the full logits. The student's log-probabilities in k_t are recomputed by the trainer, not taken from vLLM.
- vLLM's kernels give slightly different log-probabilities from the trainer's. TRL weights each episode by $\exp \sum_t (\log \pi_\theta - \log \pi_{\text{vLLM}})$ and drops episodes whose weight exceeds 3; the mean weight per stage was 0.95–1.00.
- Reward (stage-4 training, all evaluations): 1 if the last `\boxed{}` of the final answer equals the reference (numerically, to 10^{-6}), else 0; also 0 if the episode ran out of tokens or was still calling tools at the turn limit.
- Evaluation every 25 steps on 256 held-out tasks (GSM8K test questions in stage 1), one sample per task at temperature 1, since TRL has no greedy evaluation.

4 Results

Table 3 gives the first and last evaluation of every stage; Figure 1 shows all of them.

Distillation does what the derivation says. Teacher KL falls in every distillation stage, by 23–83%, and mostly within the first 25–50 steps (Qwen3, stage 2: from 0.328 to 0.049 by step 25).

Task success follows without a reward. Stages 1–3 never see the reward, yet the evaluation reward rises in each: by 0.03 and 0.13 on GSM8K, 0.53 and 0.26 on calculator tasks, 0.09 and 0.20 on three-tool tasks (Qwen3 and Qwen3.5). The students take up their teachers' tool habits. In the first batch of stage 2, the Qwen3.5 student called the calculator 4 times in 32 episodes and did the rest of the arithmetic in prose; over the last 50 steps it called it 1.55 times per episode (Qwen3: 15 times in 32 episodes, then 1.16 per episode). In stage 3, the share of executed tool calls that returned an error fell from 21% to 4% (first against last five steps; Qwen3: from 14% to 7%).

Length, not accuracy, holds Qwen3.5 back. On GSM8K, 10–18% of the Qwen3.5 student's answers run past the 1,024-token limit and score zero. They are not loops: after an awkward intermediate result, such as a fractional count, the model second-guesses itself. Of the answers that finish, 91% are right at the end of stage 1 (Qwen3: 84%). In stage 2, about 70% of its failed training episodes in the last 50 steps used all four turns re-checking a result with more calculator calls.

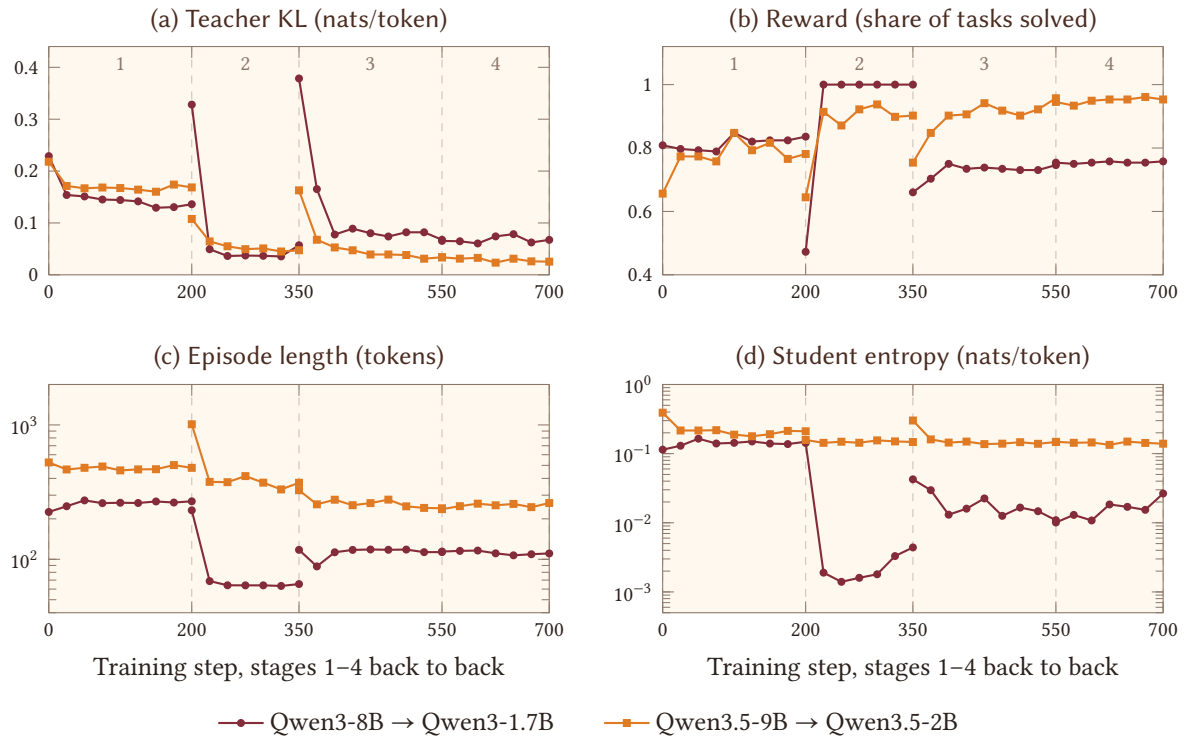


Figure 1 Evaluations during the four stages, run back to back. Dashed lines separate the stages; each stage opens with an evaluation of the previous stage's weights on the new tasks. (a) Mean of k_t over student tokens. (b) Share of the 256 held-out tasks solved. (c) Mean episode length after the prompt (log scale). (d) Mean entropy of the student's next-token distribution (log scale).

The task reward adds nothing measurable. In stage 4 the evaluation reward moves from 0.754 to 0.758 and from 0.945 to 0.953. A 256-task estimate near 0.95 has a standard error of 0.014, and the same Qwen3.5 weights scored 0.957 at the end of stage 3. Equation (6) explains why. Only 1.5% (Qwen3) and 10.9% (Qwen3.5) of the groups had any reward variance. If the four outcomes of a prompt were independent draws at the mean success rate (0.80 and 0.955), (6) would predict 59% and 17%. The Qwen3 student is far more deterministic than that: when it fails a prompt, it fails all four samples. Its entropy collapsed in stage 2 (from 0.14 to 0.002 nats per token) and stayed below 0.05; the Qwen3.5 student stayed at 0.13–0.16 from stage 2 on. Reverse KL predicts this: at its minimum the student's next-token distribution, entropy included, equals the teacher's, so a confident teacher makes a confident student (we did not measure the teachers' entropy), and a confident student gives GRPO nothing to compare. Stage 5 of the Qwen3 run, GRPO alone with $G = 8$ and no teacher, confirmed it: after the first batch every group had zero variance, the loss and gradient were exactly zero, and we stopped the run at about step 20.

Cost. The Qwen3.5 run took 7.1 hours against 2.8 (about 50 Colab compute units against 23). A token costs the same in both runs: in stage 4, a step takes 0.17 and 0.16 s per token of mean episode length. But the Qwen3.5 student writes episodes 2–5 times longer, with a sentence of explanation around every tool call (stage 2: 371 against 65 tokens; stage 3: 240 against 113).

5 Takeaways

1 Distillation is the workhorse

Prompts alone, with no labels and no reward, teach a 1.7B or 2B student to use tools: under two hours per stage on one A100, most of the gain in the first 50 steps.

2 RL needs tasks at the edge of ability

GRPO learns only from prompts whose outcome varies across samples. Distillation removes the randomness and our tasks are mostly solved; harder tasks, larger groups or a higher temperature are the levers.

3 Budgets are per model

Same configs, but Qwen3.5 wrote episodes 2–5 times longer: limits that never bound for Qwen3 cost it accuracy, and its run took 2.5 times the GPU time.

4 The tokenizer decides who can teach

The teacher must share the student's vocabulary token for token, so each family distils within itself.

Reproduction. Stages 1–4 run with

```
BACKEND=trl EXTRA=--use_vllm scripts/run_stages.sh a100 1 4
```

The trainer is `opd/trl_train.py` and the configs are in `opd/configs/`. Every logged metric, sampled episode and adapter is in `results/trl/a100/` (Qwen3) and `results/trl/a100-qwen3.5/` (Qwen3.5).

References

R. Agarwal, N. Vieillard, Y. Zhou, P. Stanczyk, S. Ramos, M. Geist, and O. Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *ICLR*, 2024.

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

Y. Gu, L. Dong, F. Wei, and M. Huang. MiniLLM: Knowledge distillation of large language models. In *ICLR*, 2024.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

Y. Kim and A. M. Rush. Sequence-level knowledge distillation. In *EMNLP*, 2016.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

K. Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. doi:10.64434/tml.20251026.

Qwen Team. Qwen3.5 model cards. Hugging Face, 2026.

S. Ross and J. A. Bagnell. Efficient reductions for imitation learning. In *AISTATS*, 2010.

S. Ross, G. J. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.

J. Schulman and Thinking Machines Lab. LoRA without regret. *Thinking Machines Lab: Connectionism*, 2025.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, S. Huang, K. Rasul, and Q. Gallouédec. TRL: Transformer reinforcement learning. https://github.com/huggingface/trl, 2020.

R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025a.

S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *ICLR*, 2025b.

Q. Yu et al. DAPO: An open-source LLM reinforcement learning system at scale. arXiv:2503.14476, 2025.